School of Future Tech

Case Study Report

on

LUHN ALGORITHM VALIDATOR

by

ARMAN CHOUDHARY

150096725075

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

In today's digital world, online transactions have become a common part of everyday life. People use credit cards, debit cards, and digital payment systems to purchase goods and services through websites and mobile applications. Because these systems deal with sensitive financial data, it is extremely important to validate user input before processing any transaction.

One of the most commonly used validation techniques for payment card numbers is the **Luhn Algorithm**. The Luhn algorithm is a simple checksum formula that helps determine whether a card number is structurally valid. It does not verify whether the card exists in a bank database, but it helps detect accidental typing errors or invalid numbers.

This case study focuses on designing and developing a **Luhn Algorithm Validator** using **HTML, CSS, and JavaScript**. The application simulates a secure payment interface where users can enter their card details and verify whether the card number is valid.

The system provides a realistic payment form with fields such as card number, expiry date, CVV, and postal code. When the user clicks the validation button, the system processes the input using JavaScript and applies the Luhn algorithm to check whether the card number is valid.

The project also demonstrates important web development concepts such as:

- Frontend form design
- Input sanitization
- Algorithm implementation in JavaScript
- Event handling and DOM manipulation
- Real-time user feedback

The main objective of this project is to build a functional validation system that demonstrates how real financial systems verify card numbers before processing transactions.

2. Problem Statement / Case Background (Abstract)

With the rapid growth of e-commerce and online payment systems, secure validation mechanisms have become essential for protecting financial transactions. When users enter card details during checkout, the system must verify that the card number follows the correct structure before sending it to a payment processor.

If a user accidentally enters an incorrect number, the system should immediately detect the error and notify the user. Without proper validation, payment systems may experience failed transactions, incorrect processing, or potential fraud risks.

The **Luhn algorithm**, developed by Hans Peter Luhn in 1954, is widely used by financial institutions and credit card companies to validate identification numbers.

Major payment networks such as:

- Visa
- MasterCard
- American Express
- Discover

use this algorithm as a preliminary validation step.

Abstract

This case study presents the design and implementation of a **web-based Luhn Algorithm Validator** that validates payment card numbers using JavaScript.

The application provides an interactive payment interface where users can enter card information and validate it instantly. The card number is processed using the Luhn checksum algorithm, which involves reversing digits, doubling alternate numbers, and calculating a checksum value.

The system also validates other fields such as expiry date and CVV to simulate a realistic checkout form. Invalid inputs are highlighted, and appropriate error messages are displayed to guide the user.

The project demonstrates how modern web technologies such as **HTML, CSS, and JavaScript** can be used together to build interactive validation systems.

3. Case Study Design

The system architecture is designed to provide a clean separation between user interface, validation logic, and user authentication.

The application contains three main modules.

3.1.User Authentication Module

A simple login system is implemented using browser LocalStorage.

The purpose of this module is to simulate user authentication before accessing the validation system. When a user logs in, the system stores login information in the browser storage. If the user is not logged in, they are redirected to the login page.

This ensures that only authenticated users can access the validator interface.

3.2.Payment Validation Interface

The main validation interface is created using HTML and CSS. It contains a structured payment form where users can input their card details.

The interface includes:

- Card number field
- Expiry date field
- CVV field
- Postal code field
- Validate payment button

The interface is designed to resemble a real payment gateway to provide a realistic user experience.

3.3.Validation and Processing Module

The validation module is implemented using JavaScript.

This module performs several tasks:

- Input sanitization
- Card number formatting
- Luhn algorithm validation
- Expiry date verification
- CVV validation
- Displaying validation results

The validation happens instantly when the user clicks the validation button.

4. Methods G Algorithms Technology Applied in the Problem Statement/ Case Study

Luhn Algorithm Explanation

The Luhn algorithm is a checksum formula used to validate identification numbers. It is commonly used for credit card numbers, government IDs, and bank account numbers.

The algorithm works as follows:

1. Remove all non-numeric characters from the card number.
2. Starting from the rightmost digit, move left.
3. Double every second digit.
4. If doubling results in a number greater than 9, subtract 9 from it.
5. Add all digits together.
6. If the total sum is divisible by 10, the card number is valid.

Example:

Card Number:
4539 1488 0343 6467

After performing the Luhn calculation, the checksum result determines whether the number is valid.

This algorithm helps detect common errors such as:

- Typing mistakes
- Incorrect digit order
- Missing digits

Technology Stack Used

HTML

HTML provides the structural foundation of the application. It defines the layout and structure of the login page and payment validation form.

HTML elements used include:

- Forms
- Input fields
- Buttons
- Navigation bar
- Modal popup

These elements allow users to interact with the application easily.

CSS

CSS is used to style the web pages and improve visual appearance.

Key styling features include:

- Modern UI layout
- Flexbox alignment
- Responsive design
- Button hover animations
- Error highlighting
- Modal success popup

CSS ensures that the interface is visually appealing and user-friendly.

JavaScript

JavaScript is responsible for the interactive functionality of the application.

The main JavaScript tasks include:

- Implementing the Luhn algorithm
- Handling form events
- Formatting card number input
- Detecting card type automatically
- Validating expiry date and CVV
- Displaying error messages dynamically

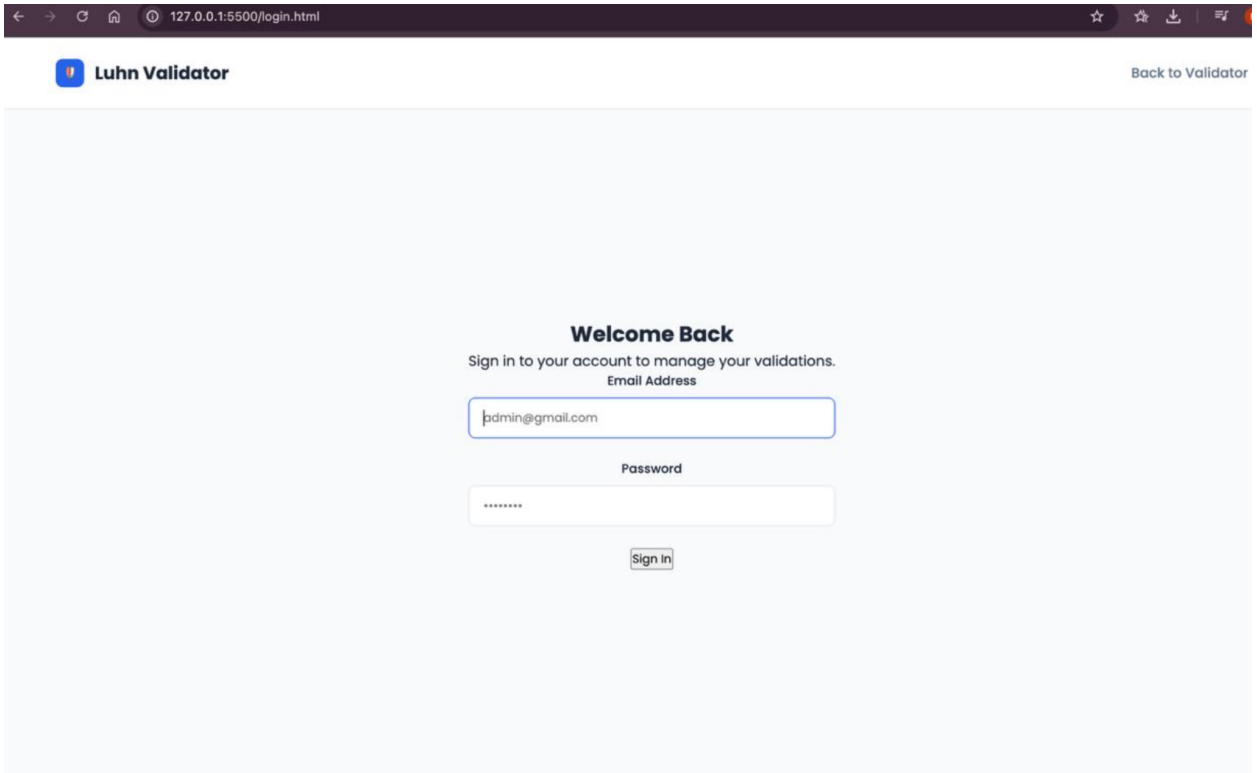
JavaScript also uses modern ES6 features such as:

- Arrow functions
- Template literals
- Fetch API
- Regular expressions

5. Problem Statement / Case Study Implementation Details and Snapshots.

The implementation consists of multiple files that work together to create the full application.

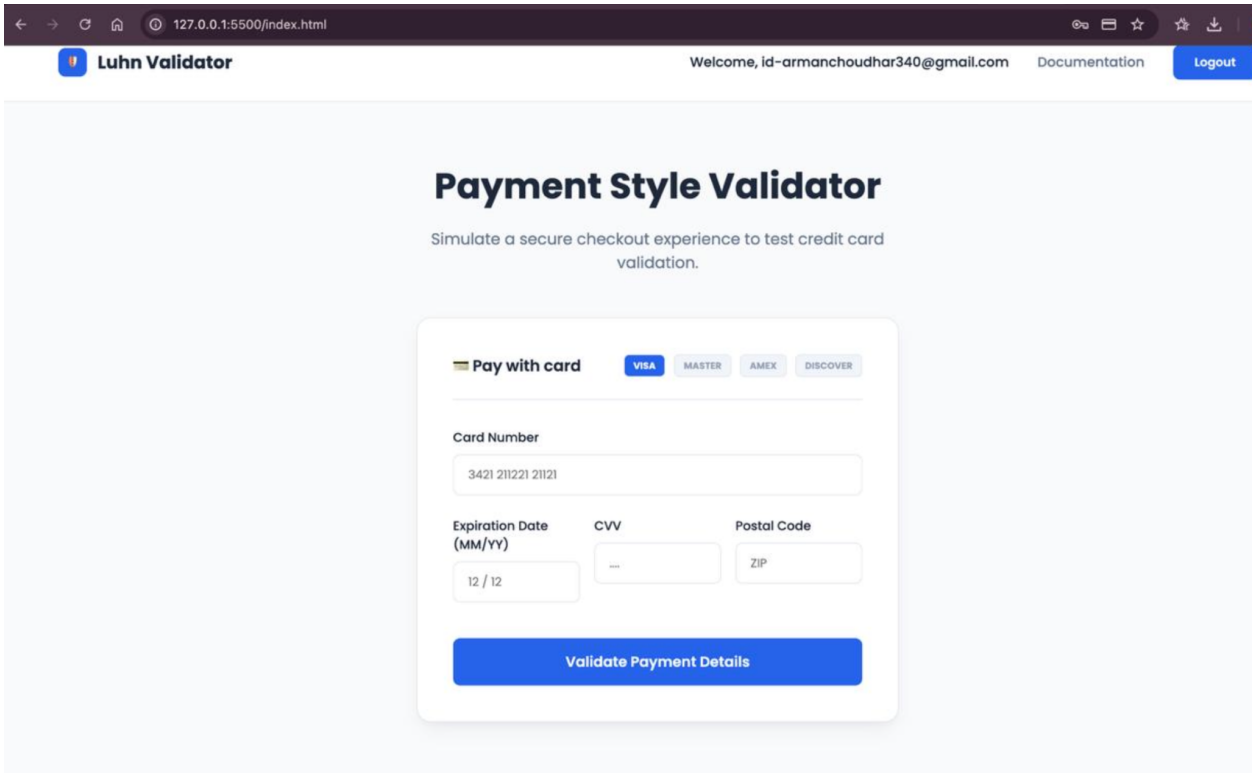
1.Login Page



This page provides the login interface for the application. Users are required to enter their email and password to access the card validation system.

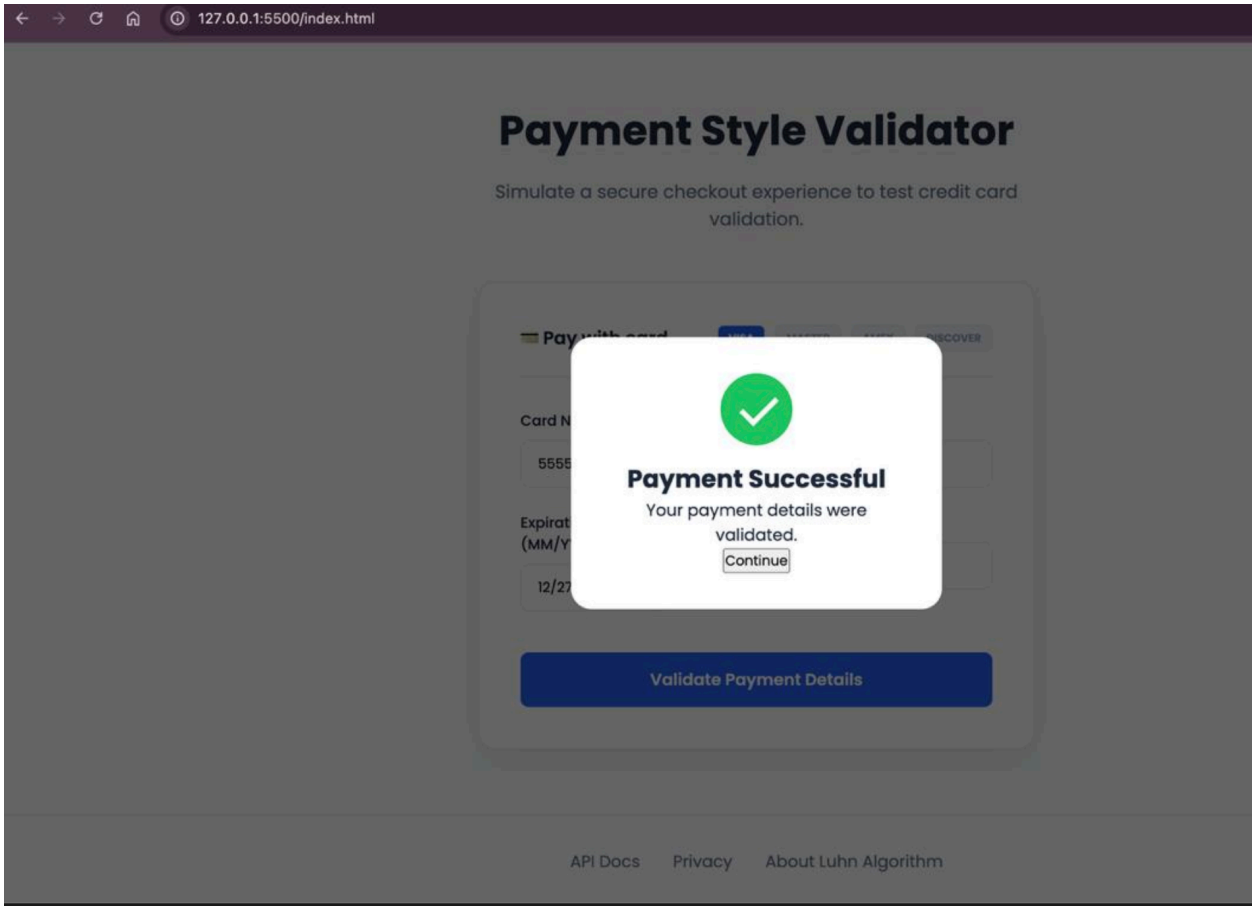
The login feature is implemented using LocalStorage, which temporarily stores the user's login status in the browser. This ensures that only authenticated users can access the payment validation page. If the user is not logged in, the system restricts access to the validator and redirects the user to the login page. This simulates a basic authentication system similar to real online payment platforms.

2. Payment Validation Page



This page contains the main card payment interface where users can enter their card details. The form includes fields such as card number, expiry date, CVV, and postal code. The interface is designed using HTML and CSS to resemble a real payment gateway. When the user clicks the "Pay with Card" or validate button, JavaScript processes the entered information and checks whether the card number is valid using the Luhn Algorithm. The system also performs additional checks such as verifying the expiry date and validating the CVV format. If any information is incorrect, the system displays an error message to guide the user.

3. Payment Success Page



This page displays a success message when the card details entered by the user pass all validation checks. Once the card number is successfully verified using the Luhn Algorithm and other fields are validated, a success modal or confirmation message is displayed on the screen. This indicates that the payment information entered by the user is structurally valid. The success message improves the user experience by providing clear feedback that the validation process has been completed successfully.

6. Problem Statement / Case Study Results and Conclusion.

Results

The developed Luhn Algorithm Validator successfully validates credit card numbers entered by users.

The system provides real-time feedback and highlights errors when incorrect details are entered.

Key results observed from the project include:

- Accurate implementation of the Luhn algorithm
- Real-time input validation
- Automatic card type detection
- Clear error messages for invalid data
- Successful display of validation results

The system demonstrates how frontend technologies can be used to implement real-world validation systems.

Conclusion

This case study demonstrates the practical implementation of a payment card validation system using the Luhn algorithm.

The project successfully combines HTML, CSS, and JavaScript to create an interactive web application that validates user inputs securely.

Through this project, important concepts such as algorithm implementation, input sanitization, DOM manipulation, and event handling were explored.

The system can be further enhanced by integrating real payment gateway APIs or server-side validation for improved security.

Overall, this project provides a solid foundation for understanding how payment systems validate card numbers in real-world financial applications.

7. References

- Mozilla Developer Network (MDN) – JavaScript Documentation
- W3Schools – HTML, CSS, and JavaScript Tutorials
- Wikipedia – Luhn Algorithm
- JavaScript Fetch API Documentation
- Web Development Form Validation Guidelines – MDN